

Universidad Complutense de Madrid
Facultad de Informática
Grado en Ingeniería de Datos e Inteligencia artificial

Proyecto - Gobernanza y Sistemas de Votación

Introducción a la Tecnología Blockchain y Smart Contracts

Vega García Camacho
Pablo Manuel Rodríguez Sosa

Curso académico: 2025/2026



1. Introducción	4
2. Descripción general del sistema	4
2.1. Funcionamiento global	4
2.2. Actores del sistema	5
2.3. Tipos de propuestas	6
3. Estructura del proyecto	6
3.1. Organización de carpetas	6
3.2. Contratos	6
3.2.1. Principales	6
3.2.2. Auxiliares	7
3.3. Contratos de prueba	7
3.3.1. quadraticVoting_test.sol	7
3.3.2. full_quadraticVoting_test.sol	8
3.3.3. security_test.sol	8
4. Diseño del contrato QuadraticVoting	8
4.1. Variables principales de estado	8
4.2. Estados de una propuesta	9
4.3. Estructura de una propuesta	9
4.4. Eventos del sistema	10
5. Ciclo de vida de una votación	10
5.1. Apertura de la votación	10
5.2. Registro de participantes	11
5.3. Baja de participantes	11
5.4. Compra adicional de tokens	11
5.5. Venta de tokens no usados	11
6. Gestión de propuestas	11
6.1. Creación de propuestas	11
6.2. Propuestas de financiación	12
6.3. Propuestas de signaling	12
6.4. Cancelación de propuestas	12
6.5. Consultas de propuestas	12
7. Sistema de votación cuadrática	13
7.1. Coste cuadrático del voto	13
7.2. Ejemplo de cálculo	13
7.3. Implementación en stake	13
7.4. Retirada de votos	13
8. Cálculo del umbral de aprobación	14
8.1. Variables utilizadas	14
8.2. Momento de recalcular el umbral	14
8.3. Condiciones de aprobación	14
9. Ejecución de propuestas externas	14
9.1. Interfaz IExecutableProposal	14
9.2. Ejecución de propuestas de financiación	15

9.3. Límite de gas	15
9.4. Ejecución de propuestas de signaling	15
10. Cierre de la votación y patrón pull-over-push	16
10.1. Función closeVoting	16
10.2. Estado de ronda cerrada	16
10.3. Reclamación de tokens	16
10.4. Ejecución de signaling	17
10.5. Cancelación de propuestas con pull-over-push	17
10.6. Preparación para una nueva votación	17
11. Token ERC20 de gobernanza	17
12. Transferencias de Ether	18
13. Seguridad y vulnerabilidades consideradas	18
13.1. Reentrancy	18
13.2. Contratos externos maliciosos	19
13.3. Manipulación de tokens	19
13.4. Control de permisos	19
13.5. Denegación de servicio	20
13.6. Contabilidad de Ether	20
13.7. Bucles restantes y funciones de consulta	20
14. Pruebas realizadas	20
14.1. Configuración de compilación	20
14.2. Organización de los tests	21
15. Decisiones de diseño relevantes	21
15.1. No reutilización de identificadores de propuestas	21
15.2. Uso de currentProposalsIds	22
15.3. Sistema de rondas	22
15.4. Participante eliminado y retirada de votos antiguos	22

1. Introducción

El proyecto se enmarca dentro del estudio de las organizaciones autónomas descentralizadas, conocidas como DAOs. En este tipo de organizaciones, las reglas de gobernanza se implementan mediante smart contracts desplegados en blockchain, lo que permite que sean públicas, verificables y resistentes a modificaciones arbitrarias.

Uno de los mecanismos más habituales de gobernanza en una DAO es la votación. A través de ella, los participantes pueden expresar sus preferencias sobre distintas propuestas y, si se cumplen determinadas condiciones, el sistema puede aprobarlas y ejecutarlas automáticamente.

El objetivo de este proyecto es implementar un sistema de votación cuadrática on-chain. En este modelo, un participante puede emitir varios votos sobre una misma propuesta, pero el coste total de esos votos crece cuadráticamente. En concreto, tener n votos sobre una propuesta cuesta n^2 tokens. Así, los participantes con más tokens pueden expresar una preferencia más intensa, pero el aumento cuadrático del coste desincentiva concentrar demasiados votos en una única propuesta.

El sistema desarrollado permite registrar participantes, comprar tokens de voto, crear propuestas, votar, retirar votos mientras las propuestas sigan pendientes, aprobar automáticamente propuestas de financiación cuando superan el umbral correspondiente y cerrar el periodo de votación devolviendo el presupuesto sobrante al propietario y habilitando que los participantes reclamen posteriormente los tokens bloqueados cuando corresponda.

Además, el sistema permite trabajar con contratos externos de propuesta. Estos contratos deben implementar la interfaz *IExecutableProposal* y soportar ERC165. Cuando una propuesta de financiación se aprueba, el contrato principal la ejecuta enviándole el presupuesto asignado. En el caso de las propuestas de signaling, no se ejecutan automáticamente dentro de *closeVoting*, sino que se ejecutan posteriormente bajo demanda mediante una función específica. Esta decisión forma parte del patrón *pull-over-push* implementado en la parte opcional.

2. Descripción general del sistema

2.1. Funcionamiento global

El sistema se organiza alrededor del contrato principal *QuadraticVoting*. Al desplegarse, este contrato crea internamente un contrato ERC20 llamado *GovernanceToken*, que se utiliza como token de voto.

El funcionamiento general del sistema es el siguiente:

1. El propietario despliega el contrato *QuadraticVoting*, indicando el precio en Wei de cada token y el número máximo de tokens que podrán emitirse.

2. El propietario abre una votación mediante *openVoting*, enviando un presupuesto inicial en Ether.
3. Los usuarios se registran como participantes mediante *addParticipant*, comprando al menos un token.
4. Los participantes pueden comprar más tokens con *buyTokens*.
5. Durante la votación, los participantes pueden crear propuestas mediante *addProposal*.
6. Los participantes votan usando stake, después de aprobar previamente al contrato *QuadraticVoting* para gastar sus tokens ERC20.
7. Si una propuesta de financiación supera el umbral de aprobación y existe presupuesto suficiente, se aprueba y ejecuta automáticamente.
8. Mientras una propuesta siga pendiente, los votantes pueden retirar votos mediante *withdrawFromProposal*.
9. El creador de una propuesta pendiente puede cancelarla mediante *cancelProposal*.
10. El propietario cierra la votación con *closeVoting*. Esta función ya no recorre todas las propuestas ni todos los votantes, sino que marca la ronda como cerrada, impide nuevas propuestas y votos, y devuelve al propietario el presupuesto sobrante.
11. Las propuestas de signaling se ejecutan bajo demanda, mediante *executeSignalingProposal*, en lugar de ejecutarse automáticamente dentro de *closeVoting*.
12. Después de procesar una propuesta de signaling, o en el caso de propuestas canceladas o de financiación no aprobadas, los participantes pueden reclamar individualmente los tokens bloqueados mediante *claimRefundFromProposal*.
13. Tras el cierre, el contrato queda preparado para abrir una nueva ronda de votación, manteniendo la posibilidad de reclamar tokens de rondas anteriores.

2.2. Actores del sistema

En el sistema intervienen los siguientes actores:

- Propietario del contrato: es la cuenta que despliega *QuadraticVoting*. Es el único actor autorizado para abrir y cerrar periodos de votación. También recibe el presupuesto no gastado al cerrar la votación.
- Participantes: son usuarios registrados mediante *addParticipant*. Pueden comprar tokens, crear propuestas, votar, retirar votos y vender tokens no utilizados.
- Creadores de propuestas: son participantes que crean propuestas durante un periodo de votación abierto. Solo el creador de una propuesta puede cancelarla, siempre que siga pendiente.
- Contratos externos de propuesta: son contratos que implementan la interfaz *IExecutableProposal* y soportan ERC165. Reciben llamadas desde *QuadraticVoting* cuando una propuesta se ejecuta.

2.3. Tipos de propuestas

El sistema distingue entre dos tipos de propuestas: propuestas de financiación y propuestas de signaling.

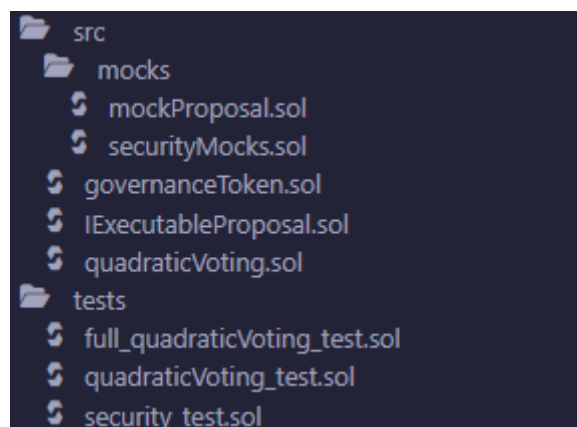
Las de financiación son propuestas con presupuesto mayor que cero. Si reciben suficientes votos y existe presupuesto disponible, pueden aprobarse automáticamente durante el periodo de votación. Al aprobarse, el sistema envía al contrato externo el presupuesto solicitado, quema los tokens usados en la votación y actualiza el presupuesto total.

Las propuestas de signaling tienen presupuesto igual a cero. Sirven para expresar preferencias o posiciones de la DAO, pero no consumen presupuesto. No se aprueban durante la votación. Una vez cerrada la ronda, pueden ejecutarse bajo demanda mediante *executeSignalingProposal*. Los tokens usados para votar propuestas de signaling no se devuelven automáticamente en *closeVoting*. Una vez cerrada la ronda, la propuesta debe procesarse mediante *executeSignalingProposal* y, después, cada votante puede reclamar individualmente sus tokens mediante *claimRefundFromProposal*.

3. Estructura del proyecto

3.1. Organización de carpetas

El proyecto está organizado en dos carpetas principales:



La carpeta src contiene los contratos principales del sistema. La carpeta tests contiene los contratos de prueba escritos para Remix.

3.2. Contratos

3.2.1. Principales

QuadraticVoting.sol contiene el contrato principal del sistema. Este contrato gestiona la apertura y cierre de votaciones, el registro de participantes, la compra y venta de tokens, la creación y cancelación de propuestas, la emisión y retirada de votos, el cálculo del umbral

de aprobación y la ejecución de propuestas externas. También se encarga de mantener la contabilidad del presupuesto de la votación mediante la variable *totalBudget*.

GovernanceToken.sol implementa el token ERC20 utilizado por el sistema de votación. Hereda de la implementación estándar ERC20 de OpenZeppelin. El contrato incluye funciones *mint* y *burn*, pero ambas están restringidas al contrato *QuadraticVoting*. De esta forma, los participantes no pueden crear ni destruir tokens arbitrariamente. Además, se define un límite máximo de emisión mediante *maxSupply*. El token redefine *decimals()* para devolver 0, de forma que una unidad del token representa exactamente un token de voto.

IExecutableProposal.sol define la interfaz que deben implementar los contratos externos de propuesta. La interfaz incluye la función: *executeProposal(uint256 proposalId, uint256 numVotes, uint256 numTokens)*. Esta función es llamada por *QuadraticVoting* cuando una propuesta se ejecuta. Además, la interfaz hereda de *IERC165*, por lo que los contratos externos deben poder declarar que soportan esta interfaz.

3.2.2. Auxiliares

MockProposal.sol es un contrato auxiliar usado para pruebas y demostraciones. No forma parte de la lógica principal de gobernanza, sino que implementa *IExecutableProposal* para comprobar que *QuadraticVoting* llama correctamente a contratos externos y transfiere el presupuesto de las propuestas de financiación.

SecurityMocks.sol también es un archivo auxiliar de pruebas. Contiene contratos diseñados para simular comportamientos adversos o no válidos, como propuestas que no implementan *IERC165*, propuestas que consumen mucho gas, propuestas de signaling que revierten y un contrato que intenta realizar *reentrancy*. Estos contratos no forman parte del sistema principal y se utilizan únicamente en *security_test.sol*.

3.3. Contratos de prueba

3.3.1. quadraticVoting_test.sol

El archivo *quadraticVoting_test.sol* contiene las pruebas funcionales principales del sistema. Para facilitar su ejecución en Remix y evitar problemas de tamaño del bytecode, las pruebas se han dividido en varios contratos pequeños, cada uno centrado en una parte concreta de la funcionalidad.

En concreto, este archivo incluye pruebas sobre el constructor, la creación del token ERC20, la apertura de votación, el registro de participantes, la compra y venta de tokens, la creación de propuestas de financiación y signaling, la votación cuadrática, la retirada de votos, la aprobación automática de propuestas de financiación, el cierre de votación con pull-over-push, la reclamación individual de tokens y las restricciones de consulta cuando la votación está cerrada.

3.3.2. full_quadraticVoting_test.sol

El archivo *full_quadraticVoting_test.sol* contiene una prueba conjunta de integración. A diferencia de las pruebas funcionales separadas, esta prueba simula un ciclo completo de uso del sistema en una única ejecución.

En esta prueba se abre una votación, se registra un participante, se crea una propuesta de financiación y una propuesta de signaling, se emiten votos cuadráticos sobre ambas, se comprueba la aprobación de la propuesta de financiación, se cierra la ronda, se ejecuta la propuesta de signaling mediante *executeSignalingProposal*, se reclaman los tokens bloqueados mediante *claimRefundFromProposal* y se comprueba que el sistema permite abrir una nueva ronda.

Esta prueba sirve para verificar que las funcionalidades principales interactúan correctamente entre sí y que el diseño *pull-over-push* funciona dentro de un flujo completo de votación.

3.3.3. security_test.sol

El archivo *security_test.sol* contiene pruebas orientadas a seguridad. Estas pruebas utilizan contratos auxiliares definidos en *src/mocks/securityMocks.sol*, que simulan comportamientos maliciosos o no válidos. Estos contratos auxiliares no forman parte del sistema principal, sino que se utilizan únicamente para comprobar la robustez de *QuadraticVoting*.

En concreto, *security_test.sol* comprueba que no se puedan añadir propuestas cuyos contratos no implementen ERC165, que una propuesta externa que consume mucho gas no bloquee el contrato principal, que el modificador *nonReentrant* proteja frente a intentos de reentrada en *sellTokens*, que una propuesta de signaling que revierte no bloquee el cierre ni la posterior reclamación de tokens, y que el contrato rechace Ether enviado directamente fuera de sus funciones controladas.

4. Diseño del contrato QuadraticVoting

4.1. Variables principales de estado

El contrato *QuadraticVoting* utiliza las siguientes variables principales:

- *owner*: dirección del propietario del contrato. Es inmutable y corresponde a la cuenta que despliega el contrato.
- *token*: instancia del contrato *GovernanceToken* creado por *QuadraticVoting*.
- *tokenPriceWei*: precio de cada token expresado en Wei.
- *totalBudget*: presupuesto disponible para financiar propuestas durante la votación.
- *votingOpen*: booleano que indica si la votación está abierta.
- *numParticipants*: número de participantes registrados.
- *numPendingProposals*: número de propuestas de financiación pendientes.
- *nextProposalId*: contador usado para asignar identificadores únicos a las propuestas.

- *currentProposalIds*: array con los identificadores de propuestas pertenecientes a la ronda de votación actual.
- *locked*: variable utilizada por el modificador *nonReentrant*.
- *currentRound*: identificador de la ronda de votación actual. Se incrementa al abrir una nueva votación y permite asociar cada propuesta a la ronda en la que fue creada.

También se utilizan varios mappings:

- *isParticipant*: indica si una dirección está registrada como participante.
- *proposals*: almacena las propuestas por identificador.
- *roundClosed*: mapping que indica qué rondas han sido cerradas. Permite saber si una propuesta antigua pertenece a una ronda ya cerrada y, por tanto, si sus tokens pueden ser reclamados.
- *userVotesInProposal*: almacena cuántos votos ha depositado cada usuario en cada propuesta. Con nuestro diseño pull-over-push, esta información también se usa para calcular los tokens que cada usuario puede reclamar individualmente.

4.2. Estados de una propuesta

Las propuestas se representan mediante el enum *ProposalStatus*, con tres posibles estados:

- Pending indica que la propuesta está activa y puede recibir votos.
- Approved indica que la propuesta de financiación ha superado el umbral y ha sido ejecutada.
- Canceled se utiliza para propuestas canceladas mediante *cancelProposal* y para propuestas de signaling ya procesadas mediante *executeSignalingProposal*. Las propuestas de financiación pendientes de una ronda cerrada no se marcan una a una como Canceled dentro de *closeVoting*, sino que quedan liquidables gracias a *roundClosed*.

4.3. Estructura de una propuesta

Cada propuesta se almacena en una estructura *Proposal* con los siguientes campos:

- *title*: título de la propuesta.
- *description*: descripción de la propuesta.
- *requiredBudget*: presupuesto solicitado. Si es cero, la propuesta se considera signaling.
- *executableContract*: dirección del contrato externo que implementa *IExecutableProposal*.
- *status*: estado de la propuesta.
- *totalVotes*: número total de votos recibidos.
- *totalTokensStaked*: número total de tokens bloqueados en la propuesta.
- *creator*: dirección del participante que creó la propuesta.

- *isSignaling*: booleano que indica si la propuesta es de signaling.
- *exists*: booleano que permite comprobar si una propuesta existe.
- *roundId*: identificador de la ronda en la que se creó la propuesta. Este campo permite que, aunque se abra una nueva votación, las propuestas de rondas anteriores sigan pudiendo liquidarse correctamente mediante reclamaciones individuales.

4.4. Eventos del sistema

El contrato emite eventos para facilitar la trazabilidad del sistema. Los eventos principales son:

- *VotingOpened*: se emite al abrir una votación e incluye el identificador de la ronda y el presupuesto inicial.
- *VotingClosed*: se emite al cerrar una votación e incluye el identificador de la ronda cerrada y el presupuesto devuelto al propietario.
- *SignalingExecutionFailed*: se emite si una propuesta de signaling falla al ejecutarse.
- *SignalingExecuted*: se emite cuando se intenta ejecutar una propuesta de signaling mediante *executeSignalingProposal*. Incluye el identificador de la propuesta y un booleano que indica si la llamada externa ha tenido éxito.
- *ParticipantAdded*: se emite al registrar un participante.
- *ParticipantRemoved*: se emite al eliminar un participante.
- *ProposalCreated*: se emite al crear una propuesta.
- *ProposalCanceled*: se emite al cancelar una propuesta.
- *VoteCast*: se emite al depositar votos.
- *VotesWithdrawn*: se emite al retirar votos.
- *ProposalApproved*: se emite al aprobar una propuesta de financiación.
- *TokensPurchased*: se emite al comprar tokens adicionales.
- *TokensSold*: se emite al vender tokens.
- *RefundClaimed*: se emite cuando un votante reclama tokens bloqueados en una propuesta mediante *claimRefundFromProposal*.

Estos eventos permiten seguir las acciones relevantes del sistema desde fuera del contrato.

5. Ciclo de vida de una votación

5.1. Apertura de la votación

La votación se abre mediante la función *openVoting*. Esta función solo puede ser llamada por el propietario del contrato gracias al modificador *onlyOwner*. Además, exige que la votación no esté ya abierta y que el propietario envíe un presupuesto inicial mayor que cero. El Ether enviado se guarda en *totalBudget* y representa el presupuesto inicial disponible para financiar propuestas.

5.2. Registro de participantes

Los usuarios se registran mediante *addParticipant*. Esta función es payable y permite comprar tokens en el momento de la inscripción. El número de tokens comprados se calcula dividiendo el Ether enviado entre *tokenPriceWei*. Se exige comprar al menos un token. Si la dirección no estaba registrada previamente, se marca como participante y se incrementa *numParticipants*. Después, el contrato acuña tokens para el participante usando *token.mint*. Si el usuario envía una cantidad de Ether que no es múltiplo exacto del precio del token, el sobrante se devuelve mediante *_refundRemainder*.

5.3. Baja de participantes

Un participante puede darse de baja mediante *removeParticipant*. Esta función requiere que *msg.sender* sea participante y utiliza *nonReentrant*. Al ejecutarse, marca la dirección como no participante y reduce el contador *numParticipants*.

Tras darse de baja, esa dirección no puede crear propuestas, votar, comprar tokens ni vender tokens mediante las funciones protegidas con *onlyParticipant*. Sin embargo, la función *withdrawFromProposal* no tiene *onlyParticipant*, por lo que un usuario eliminado todavía puede retirar votos propios de una propuesta pendiente y recuperar los tokens correspondientes.

5.4. Compra adicional de tokens

Un participante ya registrado puede comprar más tokens mediante *buyTokens*. La función exige que el usuario sea participante y que el Ether enviado alcance para comprar al menos un token. El contrato acuña nuevos tokens mediante *token.mint* y devuelve cualquier sobrante de Ether. Al finalizar, emite el evento *TokensPurchased*.

5.5. Venta de tokens no usados

Los participantes pueden vender tokens libres mediante *sellTokens*. La función exige que el usuario sea participante, que la cantidad sea mayor que cero y que tenga suficiente balance de tokens. El contrato calcula el Ether a devolver multiplicando la cantidad de tokens por *tokenPriceWei*. Después quema los tokens mediante *token.burn* y devuelve Ether al usuario usando una llamada de bajo nivel con *call*. La función está protegida con *nonReentrant* para evitar ataques de reentrada durante la devolución de Ether.

6. Gestión de propuestas

6.1. Creación de propuestas

Las propuestas se crean con *addProposal*. Esta función solo puede ejecutarse cuando la votación está abierta y por participantes registrados. Cada propuesta recibe un identificador único usando *nextProposalId*, que se incrementa después de cada creación. Además, el identificador se añade a *currentProposalIds*, para saber qué propuestas pertenecen a la ronda actual.

6.2. Propuestas de financiación

Una propuesta se considera de financiación cuando su presupuesto es mayor que cero. Estas propuestas incrementan *numPendingProposals* al crearse y pueden aprobarse automáticamente durante la votación. Cada vez que reciben votos, el contrato llama internamente a *_checkAndExecuteProposal*. Si hay presupuesto suficiente y los votos superan el umbral, la propuesta pasa a *Approved*, se ejecuta el contrato externo y se actualiza el presupuesto. Los tokens utilizados para votar una propuesta de financiación aprobada se consumen, por lo que el contrato los quema mediante *token.burn*.

6.3. Propuestas de signaling

Una propuesta se considera de signaling cuando su presupuesto es cero. Estas propuestas no incrementan *numPendingProposals* y no se aprueban durante la votación. Los tokens usados para votar una propuesta de signaling permanecen bloqueados en el contrato mientras dura la votación. En la versión con *pull-over-push*, las propuestas de signaling no se ejecutan dentro de *closeVoting*. Una vez cerrada la ronda en la que fueron creadas, pueden ejecutarse bajo demanda mediante *executeSignalingProposal*. Esta función realiza la llamada externa al contrato de propuesta con un límite de gas de 100000 unidades.

Los tokens usados para votar propuestas de signaling tampoco se devuelven automáticamente en *closeVoting*. Primero, la propuesta se procesa bajo demanda mediante *executeSignalingProposal*. Después, cada votante puede reclamar individualmente sus tokens mediante *claimRefundFromProposal*. De esta forma, *closeVoting* no depende del número de propuestas de signaling ni del número de votantes, y además se conservan *totalVotes* y *totalTokensStaked* hasta el momento de ejecutar la propuesta externa.

6.4. Cancelación de propuestas

La función *cancelProposal* permite cancelar una propuesta pendiente. Solo puede ejecutarla el creador de la propuesta y solo si la votación está abierta. Cuando se cancela una propuesta, su estado pasa a *Canceled*. Si era una propuesta de financiación, se reduce *numPendingProposals*. Además, *cancelProposal* no devuelve automáticamente los tokens a todos los votantes, ya que eso obligaría a recorrer la lista de votantes y podría producir un consumo de gas elevado. En su lugar, cada votante debe recuperar individualmente sus tokens llamando a *claimRefundFromProposal*. Esta decisión aplica el mismo patrón *pull-over-push* a la cancelación de propuestas, evitando que *cancelProposal* tenga un coste lineal respecto al número de votantes.

6.5. Consultas de propuestas

El contrato incluye varias funciones de consulta que solo pueden ejecutarse con la votación abierta, usando el modificador *whenVotingOpen*.

- *getPendingProposals*: devuelve las propuestas de financiación pendientes.
- *getApprovedProposals*: devuelve las propuestas de financiación aprobadas.
- *getSignalingProposals*: devuelve las propuestas de signaling pendientes.
- *getProposalInfo*: devuelve la información completa de una propuesta concreta.

- *getClaimableTokens*: devuelve cuántos tokens puede reclamar un votante en una propuesta concreta. A diferencia de las consultas de propuestas de la ronda actual, esta función no requiere que la votación esté abierta, ya que se utiliza precisamente para liquidar propuestas cerradas o canceladas.

7. Sistema de votación cuadrática

7.1. Coste cuadrático del voto

El sistema aplica votación cuadrática por usuario y por propuesta. El coste total de tener n votos en una propuesta es: $\text{coste total} = n^2$. Esto significa que un usuario puede emitir varios votos sobre la misma propuesta, pero cada nuevo voto aumenta el coste total de forma cuadrática. El contrato no cobra simplemente *votesToAdd*², sino la diferencia entre el nuevo coste total y el coste anterior. Esto permite votar en varias llamadas distintas respetando el coste cuadrático acumulado.

7.2. Ejemplo de cálculo

Si un usuario no tiene votos en una propuesta y añade 1 voto: $1^2 - 0^2 = 1$ token

Si después añade otro voto, pasa de 1 a 2 votos: $2^2 - 1^2 = 4 - 1 = 3$ tokens adicionales

El coste total de tener 2 votos es: $2^2 = 4$ tokens

Si un usuario pasa de 2 votos a 4 votos: $4^2 - 2^2 = 16 - 4 = 12$ tokens adicionales

Por tanto, tener 4 votos en total cuesta 16 tokens.

7.3. Implementación en stake

La función *stake* recibe el identificador de la propuesta y el número de votos que se quieren añadir.

Primero comprueba que la votación esté abierta, que el usuario sea participante, que la propuesta exista y que siga pendiente. Después obtiene los votos previos del usuario en esa propuesta, calcula los votos finales y calcula el coste incremental. El contrato transfiere los tokens desde el usuario al contrato *QuadraticVoting* usando *transferFrom*. Por tanto, antes de llamar a *stake*, el usuario debe haber hecho *approve* sobre el contrato ERC20.

Después se actualizan los votos del usuario, los votos totales de la propuesta y los tokens bloqueados. Si la propuesta es de financiación, se comprueba inmediatamente si debe aprobarse.

7.4. Retirada de votos

La función *withdrawFromProposal* permite retirar votos de una propuesta pendiente. La propuesta no puede estar aprobada ni cancelada. El contrato comprueba cuántos votos tenía el usuario y exige que pueda retirar la cantidad solicitada. Después calcula los votos finales y los tokens a devolver usando la diferencia cuadrática. Por ejemplo, retirar de 4 votos a 2 votos devuelve: $4^2 - 2^2 = 16 - 4 = 12$ tokens.

La función actualiza el estado interno y devuelve los tokens al usuario mediante *token.transfer*.

8. Cálculo del umbral de aprobación

8.1. Variables utilizadas

El enunciado define el umbral de aprobación como:

$$\text{threshold}_i = (0,2 + \text{budget}_i / \text{totalBudget}) \cdot \text{numParticipants} + \text{numPendingProposals}$$

Una propuesta de financiación se aprueba si cumple dos condiciones: el presupuesto disponible es suficiente para financiarla y el número de votos recibidos supera el umbral calculado.

En la implementación se utilizan las siguientes variables:

- *proposalBudget*: presupuesto solicitado por la propuesta.
- *totalBudget*: presupuesto total disponible en el momento de evaluación.
- *numParticipants*: número de participantes registrados en ese momento.
- *numPendingProposals*: número de propuestas de financiación pendientes.

Las propuestas de signaling no se incluyen en *numPendingProposals*.

8.2. Momento de recalcular el umbral

Solidity no trabaja con decimales de forma nativa. Por ello, el término 0,2 se representa mediante una escala por 10. Esto equivale a aplicar la fórmula del enunciado usando aritmética entera.

El contrato recalcula el umbral únicamente cuando una propuesta de financiación recibe votos. Esto ocurre al final de stake, donde se llama a *_checkAndExecuteProposal* sólo si la propuesta no es de signaling. Esto respeta la indicación del enunciado de no recalcular constantemente el umbral, sino únicamente cuando la propuesta recibe votos.

8.3. Condiciones de aprobación

La función *_checkAndExecuteProposal* comprueba primero que la propuesta esté pendiente y que no sea de signaling. Después comprueba que exista presupuesto suficiente y calcula el umbral exigiendo que los votos superen estrictamente dicho umbral. Si se cumplen las condiciones, la propuesta se marca como aprobada, se actualiza el presupuesto, se queman los tokens usados y se ejecuta el contrato externo.

9. Ejecución de propuestas externas

9.1. Interfaz IExecutableProposal

Los contratos externos de propuesta deben implementar la interfaz *IExecutableProposal*, que define la función *executeProposal*. Esta función recibe el identificador de la propuesta, el número total de votos y el número total de tokens utilizados. La función es *payable*, por lo que puede recibir Ether cuando se ejecuta una propuesta de financiación.

Esta interfaz hereda de IERC165. Esto permite que el contrato principal verifique si un contrato externo declara soporte para la interfaz antes de aceptarlo como propuesta. Si el contrato externo no implementa la interfaz, la propuesta no se crea.

9.2. Ejecución de propuestas de financiación

Las propuestas de financiación se ejecutan automáticamente cuando superan el umbral y hay presupuesto suficiente.

Durante la ejecución:

1. La propuesta pasa a estado *Approved*.
2. Se reduce *numPendingProposals*.
3. Se calcula el equivalente en Ether de los tokens usados.
4. Se actualiza *totalBudget*.
5. Se queman los tokens bloqueados.
6. Se llama al contrato externo mediante *call*, enviando el presupuesto solicitado.

La llamada externa incluye el identificador de propuesta, los votos totales y los tokens usados.

9.3. Límite de gas

Todas las llamadas externas a contratos de propuesta se realizan con un límite máximo de 100,000 unidades de gas. Este límite se aplica tanto a las propuestas de financiación, ejecutadas cuando superan el umbral de aprobación, como a las propuestas de signaling, ejecutadas después del cierre mediante *executeSignalingProposal*. El objetivo de este límite es evitar que un contrato externo de propuesta pueda consumir todo el gas disponible de la transacción y provocar un bloqueo o un consumo excesivo.

9.4. Ejecución de propuestas de signaling

Las propuestas de signaling se ejecutan después del cierre de la ronda, pero no dentro de *closeVoting*. Para ello se utiliza la función *executeSignalingProposal*, que puede ser llamada una vez la ronda de la propuesta está cerrada.

Como las propuestas de signaling no tienen presupuesto asociado, la llamada externa se realiza sin enviar Ether. Aun así, se aplica el mismo límite máximo de 100000 unidades de gas que en las propuestas de financiación.

Antes de realizar la llamada externa, la propuesta pasa a estado Canceled. Esto evita que pueda ejecutarse más de una vez. Después de la llamada, el contrato emite el evento *SignalingExecuted*, indicando el identificador de la propuesta y si la ejecución ha tenido éxito. Si la ejecución externa falla, también se emite *SignalingExecutionFailed*. Como la ejecución de signaling está separada de *closeVoting*, una propuesta maliciosa no puede impedir el cierre de la ronda ni bloquear la devolución individual de tokens.

10. Cierre de la votación y patrón pull-over-push

10.1. Función closeVoting

La función *closeVoting* solo puede ser ejecutada por el propietario y solo cuando la votación está abierta. Además, está protegida con *nonReentrant*.

closeVoting implementa el patrón *pull-over-push*. Esto significa que la función no devuelve tokens a todos los votantes ni ejecuta todas las propuestas de signaling directamente. En lugar de ello, *closeVoting* se limita a cerrar la ronda actual, marcarla como cerrada y devolver al propietario el presupuesto sobrante.

Esta decisión evita que el cierre de la votación dependa del número de propuestas o del número de votantes, reduciendo el riesgo de denegación de servicio por consumo excesivo de gas.

10.2. Estado de ronda cerrada

El contrato utiliza *currentRound* para identificar la ronda actual y *roundClosed* para registrar qué rondas ya han sido cerradas. Cada propuesta almacena el identificador de la ronda en la que fue creada mediante *roundId*. Cuando *closeVoting* se ejecuta, la ronda actual se marca como cerrada. A partir de ese momento, ya no se pueden crear propuestas ni votar en esa ronda, pero sí se permiten las operaciones de liquidación asociadas a ella. Este diseño permite que los participantes recuperen tokens de propuestas antiguas incluso si el propietario ya ha abierto una nueva ronda de votación.

10.3. Reclamación de tokens

Los tokens bloqueados en propuestas no aprobadas no se devuelven automáticamente dentro de *closeVoting*. En su lugar, cada votante puede llamar a *claimRefundFromProposal* para recuperar los tokens que le correspondan. Esta función permite reclamar tokens en propuestas canceladas y en propuestas de financiación que no fueron aprobadas antes del cierre de su ronda. En el caso de las propuestas de signaling, la reclamación se permite una vez que la propuesta ha sido procesada mediante *executeSignalingProposal*.

La cantidad reclamable se calcula a partir de los votos que el usuario tenía depositados en la propuesta. Una vez realizada la reclamación, se actualiza el estado interno para evitar que el mismo usuario pueda reclamar dos veces los mismos tokens. Además, se reducen los contadores *totalVotes* y *totalTokensStaked* de la propuesta, de forma que estos valores reflejan los votos y tokens que siguen pendientes de liquidar.

10.4. Ejecución de signaling

Las propuestas de signaling tampoco se ejecutan automáticamente dentro de *closeVoting*. Una vez cerrada la ronda correspondiente, la ejecución puede realizarse mediante *executeSignalingProposal*. Esta función realiza la llamada al contrato externo de propuesta con un límite de 100000 unidades de gas.

Antes de realizar la llamada externa, la propuesta se marca como Canceled para impedir que pueda procesarse más de una vez. En la llamada se envían al contrato externo los valores *totalVotes* y *totalTokensStaked* acumulados durante la votación. Por este motivo, los votantes no pueden reclamar los tokens de una propuesta de signaling pendiente antes de que esta haya sido procesada con *executeSignalingProposal*.

Si la ejecución externa falla, se emite el evento *SignalingExecutionFailed*, pero la propuesta queda igualmente procesada y los votantes pueden reclamar después sus tokens mediante *claimRefundFromProposal*. Separar la ejecución de signaling del cierre de la votación evita que una propuesta externa maliciosa pueda bloquear *closeVoting*.

10.5. Cancelación de propuestas con pull-over-push

La función *cancelProposal* también sigue el patrón *pull-over-push*. Cuando el creador cancela una propuesta pendiente, el contrato solo cambia su estado a Canceled y actualiza los contadores necesarios. Los tokens no se devuelven automáticamente a todos los votantes. Cada votante debe recuperarlos mediante *claimRefundFromProposal*. Esto evita que *cancelProposal* tenga que recorrer todos los votantes de la propuesta.

10.6. Preparación para una nueva votación

Después de cerrar una votación, *votingOpen* queda en false y el propietario puede abrir una nueva ronda mediante *openVoting*. El contrato no reinicia *nextProposalId*, por lo que los identificadores de propuestas no se reutilizan. Además, el uso de *roundId* y *roundClosed* permite distinguir propuestas de distintas rondas y liquidar correctamente propuestas antiguas incluso después de abrir una nueva votación.

11. Token ERC20 de gobernanza

El contrato *GovernanceToken* hereda de la implementación ERC20 de *OpenZeppelin*. De esta forma, se reutiliza una implementación estándar y probada del token ERC20, sin modificar su lógica interna.

La creación de tokens se realiza mediante la función *mint*. Solo el contrato *QuadraticVoting* puede llamar a esta función, gracias al modificador *onlyVoting*. Los tokens se crean cuando un usuario se registra con *addParticipant* o compra tokens adicionales con *buyTokens*.

La eliminación de tokens se realiza mediante la función *burn*. También está restringida al contrato *QuadraticVoting*. El sistema quema tokens en dos casos: cuando un participante vende tokens con *sellTokens* o cuando una propuesta de financiación se aprueba y los tokens usados en sus votos se consumen.

El contrato define una variable inmutable *maxSupply*, que indica el máximo número de tokens que pueden emitirse. Antes de acuñar nuevos tokens, se comprueba cuantos tokens hay en el momento para impedir que se emitan más tokens de los permitidos en el constructor de *QuadraticVoting*.

El contrato redefine la función *decimals* para devolver 0. Esta decisión simplifica el sistema, porque un token representa exactamente una unidad de voto. Así se evita trabajar con fracciones de token.

El contrato *QuadraticVoting* no mueve tokens de los participantes sin autorización. En la función *stake*, los tokens se transfieren desde el participante al contrato mediante *transferFrom*. Esto exige que el participante haya aprobado previamente al contrato *QuadraticVoting* mediante la función *approve* del ERC20. De esta forma, se respeta la propiedad de los tokens y se sigue el funcionamiento estándar de ERC20.

12. Transferencias de Ether

El contrato *QuadraticVoting* utiliza llamadas de bajo nivel con *call* en las operaciones donde debe enviar Ether, por ejemplo, para:

- devolver el sobrante de Ether cuando un usuario envía una cantidad que no es múltiplo exacto del precio del token
- devolver Ether a un participante cuando vende tokens mediante *sellTokens*
- enviar el presupuesto a un contrato externo cuando se aprueba una propuesta de financiación
- realizar llamadas externas a propuestas de signaling mediante *executeSignalingProposal*; en este caso se usa *call*, pero no se envía Ether porque su presupuesto es cero.
- devolver al propietario el presupuesto sobrante al cerrar la votación

En todos los casos en los que se transfiere Ether se comprueba si la llamada ha tenido éxito. Esta decisión evita usar *send* o *transfer*, que limitan el gas disponible para el receptor. El único contrato del sistema que realiza transferencias de Ether mediante llamadas de bajo nivel es *QuadraticVoting*. El token ERC20 no gestiona Ether, sino únicamente balances de tokens.

El contrato incluye una función *receive* que revierte si alguien intenta enviar Ether directamente. Esta decisión evita que entre Ether al contrato fuera de las funciones

controladas del sistema. Así se impide que haya Ether no contabilizado en *totalBudget* o no asociado a la compra de tokens.

13. Seguridad y vulnerabilidades consideradas

13.1. Reentrancy

El contrato implementa un modificador *nonReentrant* basado en una variable booleana *locked*.

Este modificador se aplica en funciones sensibles como:

- *addParticipant*;
- *removeParticipant*;
- *addProposal*;
- *cancelProposal*;
- *buyTokens*;
- *sellTokens*;
- *stake*;
- *withdrawFromProposal*;
- *closeVoting*.
- *claimRefundFromProposal*
- *executeSignalingProposal*

Esto protege especialmente las funciones que realizan transferencias de Ether o tokens, o que modifican el estado interno del sistema.

13.2. Contratos externos maliciosos

El sistema interactúa con contratos externos de propuesta. Para reducir riesgos, se aplican varias medidas:

1. *addProposal* verifica mediante ERC165 que el contrato externo soporta *IExecutableProposal*.
2. Las llamadas externas a propuestas se realizan con un límite de 100000 gas.
3. En el caso de propuestas de signaling, la llamada externa no se realiza dentro de *closeVoting*, sino mediante *executeSignalingProposal*. Si la llamada falla, se emite un evento *SignalingExecutionFailed*. Como la ejecución de signaling está separada del cierre, una propuesta maliciosa no puede bloquear *closeVoting* ni impedir que los participantes reclamen sus tokens.

13.3. Manipulación de tokens

Los tokens solo pueden ser creados o destruidos por el contrato *QuadraticVoting*. Esto se implementa en *GovernanceToken* mediante el modificador *onlyVoting*. Además, existe un máximo de tokens emitibles definido por *maxSupply*. En la votación, el contrato no toma

tokens de los usuarios directamente sin permiso. Para votar, el usuario debe aprobar previamente al contrato mediante `approve`, y luego *QuadraticVoting* usa `transferFrom`.

13.4. Control de permisos

El sistema utiliza distintos modificadores para controlar permisos:

- *onlyOwner*: restringe apertura y cierre de votación al propietario.
- *onlyParticipant*: restringe operaciones de participación a usuarios registrados.
- *whenVotingOpen*: impide ejecutar ciertas funciones cuando la votación está cerrada.

También existen comprobaciones específicas, como en *cancelProposal*, donde solo el creador de la propuesta puede cancelarla.

13.5. Denegación de servicio

Uno de los riesgos principales del sistema es la denegación de servicio por consumo excesivo de gas. Si *closeVoting* tuviera que recorrer todas las propuestas y todos los votantes para devolver tokens y ejecutar propuestas de signaling, un atacante podría intentar crear muchos datos de usuario para que el cierre no cupiera en una sola transacción.

Para evitar este problema, en lugar de hacer todas las operaciones dentro de *closeVoting*, el contrato separa el cierre de la votación de las operaciones de liquidación. *closeVoting* solo marca la ronda como cerrada y devuelve el presupuesto sobrante al propietario. Posteriormente, las propuestas de signaling se procesan individualmente mediante *executeSignalingProposal* y, después, cada usuario reclama sus tokens mediante *claimRefundFromProposal*.

También se aplica este patrón a *cancelProposal*, que no devuelve tokens recorriendo votantes, sino que habilita a cada votante a reclamar sus tokens individualmente. Además, las llamadas externas a contratos de propuesta se realizan con un límite de 100000 unidades de gas.

13.6. Contabilidad de Ether

La variable *totalBudget* representa el presupuesto disponible para financiar propuestas. El presupuesto aumenta cuando se aprueba una propuesta de financiación, sumando el equivalente en Ether de los tokens usados en los votos. Y disminuye al enviar fondos a la propuesta aprobada. El contrato rechaza Ether enviado directamente mediante `receive`, lo que evita entradas de Ether no contabilizadas. Además, al cerrar la votación, el presupuesto sobrante se devuelve al propietario.

13.7. Bucles restantes y funciones de consulta

Con el patrón *pull-over-push*, las funciones críticas del sistema ya no dependen de bucles sobre datos de usuario para cerrar la votación, devolver tokens o ejecutar propuestas de signaling.

Siguen existiendo bucles en funciones de consulta, como las que devuelven propuestas pendientes, aprobadas o de signaling. Sin embargo, estas funciones son view y no realizan transferencias de Ether, transferencias de tokens ni llamadas externas. Por tanto, no bloquean el cierre de la votación ni la recuperación individual de tokens.

14. Pruebas realizadas

14.1. Configuración de compilación

El proyecto se ha compilado en Remix usando la siguiente configuración:

Solidity compiler: 0.8.20

Optimizer: enabled

Runs: 1

Los tests se han dividido en varios contratos pequeños para evitar que los contratos de prueba superen el límite de tamaño de bytecode de la EVM. Remix compila los tests Solidity como contratos desplegados, por lo que un único contrato de test demasiado grande generó al principio warnings de tamaño aunque el contrato principal no los generaba, por eso se tomó esta decisión.

14.2. Organización de los tests

Las pruebas se encuentran en dos archivos principales:

- *quadraticVoting_test.sol* contiene pruebas funcionales del sistema.
- *security_test.sol* contiene pruebas específicas de seguridad y contratos auxiliares maliciosos.

Los contratos de prueba están divididos por funcionalidad, por ejemplo, tests de cierre, tests de participantes, tests de propuestas, tests de segunda ronda, tests de seguridad frente a reentrancy y tests de verificación ERC165. Por otro lado, los contratos de test que necesitan enviar Ether reciben fondos mediante la anotación *#value* de Remix, lo que permite probar funciones payable como *openVoting*, *addParticipant* y *buyTokens*.

Además de las pruebas unitarias de cada funcionalidad, se incluye una prueba conjunta que simula un ciclo completo de votación en un archivo auxiliar (*full_quadraticVoting_test.sol*). En esta prueba se abre una votación, se registra un participante, se crean propuestas, se emiten votos cuadráticos, se aprueba una propuesta de financiación, se cierra la ronda, se ejecuta una propuesta de signaling mediante *pull-over-push* y se reclaman tokens

bloqueados. Con esto se comprueba que las funcionalidades principales interactúan correctamente entre sí.

15. Decisiones de diseño relevantes

15.1. No reutilización de identificadores de propuestas

El contrato no reinicia *nextProposalId* al cerrar una votación. Esto evita reutilizar identificadores de propuestas en rondas posteriores. Esta decisión es importante porque existen mappings anidados, como *userVotesInProposal*, que no pueden borrarse completamente de forma sencilla para todos los usuarios. Si se reutilizara un identificador antiguo, podrían aparecer votos previos asociados a una propuesta nueva. Por tanto, los identificadores de propuestas son únicos y crecientes a lo largo de toda la vida del contrato.

15.2. Uso de *currentProposalsIds*

Aunque los identificadores de propuestas no se reutilizan, el contrato necesita saber qué propuestas pertenecen a la ronda actual. Para ello usa el array *currentProposalsIds*. Cada vez que se crea una propuesta, su identificador se añade a este array.

Este array permite implementar funciones de consulta sobre las propuestas de la ronda actual, como *getPendingProposals*, *getApprovedProposals* y *getSignalingProposals*.

Al cerrar una ronda, el array se elimina para preparar la siguiente votación. Las propuestas antiguas siguen existiendo en el mapping *proposals* y pueden liquidarse gracias a su *roundId*, pero ya no forman parte de las consultas de la ronda actual.

15.3. Sistema de rondas

Para implementar *pull-over-push* y permitir varias votaciones sucesivas, el contrato usa un sistema de rondas. *currentRound* identifica la ronda de votación actual. Cada propuesta guarda en su campo *roundId* la ronda en la que fue creada. Cuando se cierra una votación, *roundClosed* marca esa ronda como cerrada.

Este diseño permite que las propuestas antiguas sigan siendo liquidables aunque se abra una nueva ronda. Por ejemplo, un usuario puede reclamar tokens bloqueados en una propuesta de una ronda anterior mediante *claimRefundFromProposal* incluso si ya existe una nueva votación abierta. Sin este sistema de rondas, abrir una nueva votación podría dificultar distinguir qué propuestas pertenecen a rondas cerradas y cuáles pertenecen a la ronda activa.

15.4. Participante eliminado y retirada de votos antiguos

Cuando un participante se elimina mediante *removeParticipant*, deja de poder votar, crear propuestas, comprar tokens o vender tokens, porque esas funciones requieren *onlyParticipant*. Sin embargo, *withdrawFromProposal* no exige que el usuario siga siendo

participante. Esto permite que un usuario eliminado pueda retirar votos que había depositado anteriormente, siempre que la propuesta siga pendiente. Esta decisión evita que los tokens de un usuario queden bloqueados simplemente por haberse dado de baja como participante.